

[BUG] Adjoint and backpropagation differentiation gives different results f

<https://github.com/PennyLaneAI/pennylane/issues/2434>

ROW 20

[BUG] Adjoint and backpropagation differentiation gives different results for custom qml.Hermitian observables #2434

New issue

Closed

#2543



vbelis opened on Apr 11, 2022

Expected behavior

A QML model with a custom `qml.Hermitian` observable should give the identical results when trained with either adjoint differentiation or backprop differentiation methods.

Actual behavior

The two differentiation methods give different results:

- The backprop trained algorithm learns and produces the expected accuracy.
- The adjoint trained algorithm does not learn at all, final predictions for a binary data set are always 0.5.

Again, as per the documentation, the two algorithms should produce identical results.

Additional information

Here is the output produced by training with backprop and adjoint, respectively.

[backprop_output.txt](#)
[adjoint_output.txt](#)

If we change to a default observable (line 35-36 in the code below) such as PauliZ we observe that adjoint and backprop give identical results, as you can see below.

[adjoint_output_pauliZ.txt](#)
[backprop_output_pauliZ.txt](#)

This issue was first discussed here: <https://discuss.pennylane.ai/t/adjoint-and-backpropagation-differentiation-methods-give-different-results/1731>.

Source code

```
# Minimal example that reproduces the issue for the backprop and adjoint discrepancy.
import torch.nn as nn
import numpy as np
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt
import torch
import pennylane as qml
import sys
from time import perf_counter

class Model(nn.Module):
    def __init__(self, dev, diff_method="backprop"):
        super().__init__()

        self.cnet_in = self.get_cnet_in()
        self.qcircuit = qml.QNode(dev, interface="torch",
                                   diff_method=diff_method)(self.qnode)

        weight_shape = {"weights":(2,)}
        self.qlayer = qml.qnn.TorchLayer(self.qcircuit, weight_shape)

    def get_cnet_in(self):
        layers = [nn.Linear(2,16), nn.ReLU(True), nn.Linear(16,2), nn.Sigmoid()]
        return nn.Sequential(*layers)

    def qnode(self, inputs, weights):
        # Data encoding:
        for x in range(len(inputs)):
            qml.Hadamard(x)
            qml.RZ(2.0 * inputs[x], wires=x)
```

Assignees

No one assigned

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Fix 'diff_method="adjoint"' bug with 'qml.Hermitian' (inc. adding 'tape.get_parameters' 'operations_only' arg)
PennyLaneAI/pennylane

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants

5 participants +1

```

diff_method=diff_method)(self.qnode)

weight_shape = {"weights":(2,)}
self.qlayer = qml.qnn.TorchLayer(self.qcircuit, weight_shape)

def get_cnet_in(self):
    layers = [nn.Linear(2,16), nn.ReLU(True), nn.Linear(16,2), nn.Sigmoid()]
    return nn.Sequential(*layers)

def qnode(self, inputs, weights):
    # Data encoding:
    for x in range(len(inputs)):
        qml.Hadamard(x)
        qml.RZ(2.0 * inputs[x], wires=x)
    # Trainable part:
    qml.CNOT(wires=[0,1])
    qml.RY(weights[0], wires=0)
    qml.RY(weights[1], wires=1)
    o = [[1], [0]] * np.conj([[1], [0]]).T
    return qml.expval(qml.Hermitian(o, wires=[0]))
    #return qml.expval(qml.PauliY(wires=0))

def forward(self, x):
    x = self.cnet_in(x)
    x = self.qlayer(x)
    return x

def train(X, y, dev_name, diff_method):

    dev = qml.device(dev_name, wires=2, shots=None)
    model = Model(dev, diff_method)

    # Train the model
    opt = torch.optim.Adam(model.parameters(), lr=0.01)
    loss = torch.nn.MSELoss()

    X = torch.tensor(X, requires_grad=False).float()
    y = torch.tensor(y).float()

    batch_size = 5
    batches = 200 // batch_size

    data_loader = torch.utils.data.DataLoader(
        list(zip(X, y)), batch_size=batch_size, shuffle=True, drop_last=True
    )

    epochs = 20
    avg_loss = []
    for epoch in range(epochs):

        running_loss = 0
        x = 0

        for xs, ys in data_loader:
            opt.zero_grad()

            loss_evaluated = loss(model(xs), ys)
            loss_evaluated.backward()
            opt.step()
            #print('Weights:', model.state_dict()['qlayer.weights'])
            running_loss += loss_evaluated
        loss_value = running_loss / batches
        avg_loss.append(loss_value.detach().numpy())
        print("Average loss over epoch {}: {:.10f}".format(epoch + 1, loss_value))

    y_pred = model(X)
    print(y_pred)
    predictions = (y_pred >= 0.5).type(torch.uint8)
    correct = [1 if p == p_true else 0 for p, p_true in zip(predictions, y)]
    accuracy = sum(correct) / len(correct)
    print(f"Accuracy: {accuracy * 100}%")

    return avg_loss

if __name__ == "__main__":
    torch.manual_seed(42)
    np.random.seed(42)
    X, y = make_moons(200)

```

Tracebacks

No response

System information

```
WARNING: pip is being invoked by an old script wrapper. This will fail in a future version of pip.
Please see https://github.com/pypa/pip/issues/5599 for advice on fixing the underlying issue.
To avoid this problem you can invoke Python with '-m pip' instead of running pip directly.
Name: PennyLane
Version: 0.20.0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author:
Author-email:
License: Apache License 2.0
Location: /home/vabelis/anaconda3/envs/ae_qml_pnl/lib/python3.8/site-packages
Requires: scipy, semantic-version, autograd, autoray, appdirs, pennylane-lightning, toml, numpy, cachetools, r
Required-by: PennyLane-qiskit, PennyLane-Lightning
Platform info: Linux-3.10.0-1160.36.2.el7.x86_64-x86_64-with-glibc2.17
Python version: 3.8.12
Numpy version: 1.22.2
Scipy version: 1.7.3
Installed devices:
- default.gaussian (PennyLane-0.20.0)
- default.mixed (PennyLane-0.20.0)
- default.qubit (PennyLane-0.20.0)
- default.qubit.autograd (PennyLane-0.20.0)
- default.qubit.jax (PennyLane-0.20.0)
- default.qubit.tf (PennyLane-0.20.0)
- default.qubit.torch (PennyLane-0.20.0)
- qiskit.aer (PennyLane-qiskit-0.21.0)
- qiskit.basicaer (PennyLane-qiskit-0.21.0)
- qiskit.ibmq (PennyLane-qiskit-0.21.0)
- qiskit.ibmq.circuit_runner (PennyLane-qiskit-0.21.0)
- qiskit.ibmq.sampler (PennyLane-qiskit-0.21.0)
- lightning.qubit (PennyLane-Lightning-0.22.1)
```

Existing GitHub issues

☒ I have searched existing GitHub issues to make sure the issue does not already exist.



 **vbelis** added **bug** 🐛 on Apr 11, 2022



CatalinaAlbornoz on Apr 11, 2022

Contributor ...

Hi [@vbelis](#), thank you for posting this issue here and for going deep into the problem in the forum. This will help us tackle the issue.



trbromley on Apr 13, 2022 · edited by trbromley

Edits ▾ Contributor ...

I agree that this is a bug with the adjoint method gradient.

Here is a small code block that highlights the issue:

```
import pennylane as qml
from pennylane import numpy as np

dev = qml.device("default.qubit", wires=2)

def circuit(weights):
    qml.RY(weights[0], wires=0)
    qml.RY(weights[1], wires=1)
    return qml.expval(qml.Hermitian(qml.PauliZ.compute_matrix(), wires=[0])) # doesn't work with adjoint
# return qml.expval(qml.PauliZ(0)) # works with adjoint
```

```
weights = np.ones(2, requires_grad=True)

qnode_backprop = qml.QNode(circuit, dev, diff_method="backprop", interface="autograd")
qnode_adjoint = qml.QNode(circuit, dev, diff_method="adjoint", interface="autograd")

grad_backprop = qml.grad(qnode_backprop)(weights)
grad_adjoint = qml.grad(qnode_adjoint)(weights)

print(f"Backprop gradient: {grad_backprop}") # [-8.41470985e-01  8.32667268e-17]
print(f"Adjoint gradient: {grad_adjoint}") # [ 0.          -0.84147098]
print(f"Gradients are the same: {np.allclose(grad_backprop, grad_adjoint)}") # False
```

Note that:

- This is an issue for all interfaces, not just `torch`
- The problem is specific to returning the expectation value of a `qml.Hermitian()` object
- `lightning.qubit` doesn't support `qml.Hermitian` with the adjoint method, so this is less of a problem there but still good to be aware

Drilling down a bit more, the problem stems from line 1037 in the `adjoint_jacobian()` method. When we have `return qml.expval(qml.Hermitian(qml.PauliZ.compute_matrix(), wires=[0]))` then `param_number = 3`, but when we have `return qml.expval(qml.PauliZ(0))` then `param_number = 2` (as expected). You can see this more succinctly with the following:

```
# option 1
with qml.tape.QuantumTape() as tape:
    qml.RY(weights[0], wires=0)
    qml.RY(weights[1], wires=1)
    qml.expval(qml.Hermitian(qml.PauliZ.compute_matrix(), wires=[0]))

print(tape.num_params) # this is 3

# option 2
with qml.tape.QuantumTape() as tape:
    qml.RY(weights[0], wires=0)
    qml.RY(weights[1], wires=1)
    qml.expval(qml.PauliZ(0))

print(tape.num_params) # this is 2
```

This leads to a related question:

Should Hermitian matrices count as trainable parameters in a quantum tape?



josh146 on Apr 13, 2022

Member ...

| Should Hermitian matrices count as trainable parameters in a quantum tape?

@trbromley is the bug addressed by specifying `requires_grad=False` for the Hermitian matrix? E.g.,

```
qml.expval(np.array(..., requires_grad=False))
```

If so, this may be intended behaviour in the Autograd interface (albeit unintuitive behaviour)



trbromley on Apr 13, 2022 · edited by trbromley

Edits ▾ Contributor ...

Hey @josh146, I think it's still a problem:

```
import pennylane as qml
from pennylane import numpy as np

obs = np.array(qml.PauliZ.compute_matrix(), requires_grad=False)

with qml.tape.QuantumTape() as tape:
    qml.RY(0.1, wires=0)
    qml.RY(0.2, wires=1)
```

Hey @josh146, I think it's still a problem:

```
import pennylane as qml
from pennylane import numpy as np

obs = np.array(qml.PauliZ.compute_matrix(), requires_grad=False)

with qml.tape.QuantumTape() as tape:
    qml.RY(0.1, wires=0)
    qml.RY(0.2, wires=1)
    qml.expval(qml.Hermitian(obs, wires=[0]))

tape.num_params # this is 3
```

And we still get 3 in the `torch` and `tf` interfaces.



josh146 on Apr 13, 2022

Member ...



josh146 on Apr 13, 2022 · edited by josh146

Edits Member ...

Oh @trbromley `requires_grad` is only taken into account when using `qml.execute`. So this will work when using a `QNode`:

```
import pennylane as qml
from pennylane import numpy as np

obs = np.array(qml.PauliZ.compute_matrix(), requires_grad=False)
dev = qml.device("default.qubit", wires=2)

@qml.qnode(dev)
def f(x):
    qml.RY(x[0], wires=0)
    qml.RY(x[1], wires=1)
    return qml.expval(qml.Hermitian(obs, wires=[0]))

>>> f(np.array([0.1, 0.2]))
tensor(0.99500417, requires_grad=True)
>>> f.tape.num_params
2
```

This is because interfaces are not used until execution. Long term, I've always thought we could move this to tape initialization.



1

trbromley on Apr 13, 2022

Contributor ...

So maybe in `adjoint_jacobian()` we need to count how many parameters have `requires_grad=True`? Although maybe that doesn't work for all interfaces 😞



josh146 on Apr 13, 2022

Member ...

I think `adjoint_jacobian` should be getting the tape after `qml.execute()` has *already* determined the trainable parameters, so inside this method, `tape.num_params` should be correct. If not, this would be a bug





rmoyard on Apr 14, 2022

Contributor ...

Hi @vbelis, Could you try to replace to add `requires_grad=False` to the matrix `o = [[1], [0]] * np.conj([[1], [0]]).T` by using `o = np.array(o, requires_grad=False)` and let us know if it solves the problem, thank you in advance!



vbelis on Apr 19, 2022

Author ...

Hi @trbromley, @CatalinaAlbornoz, @rmoyard, @josh146, thanks for your fast replies and useful information! I tried running the posted code using `pennylane.numpy`, and adding:

```
o = [[1], [0]] * np.conj([[1], [0]]).T
o = np.array(o, requires_grad=False)
return qml.expval(qml.Hermitian(o, wires=[0]))
```



Adjoint and backprop give different results. I also get the following warning for having Autograd and Torch tensors:

```
/lib/python3.8/site-packages/pennylane/math/multi_dispatch.py:63: UserWarning: Contains tensors of types
{'torch', 'autograd'}; dispatch will prioritize TensorFlow and PyTorch over autograd. Consider replacing
Autograd with vanilla NumPy.
warnings.warn(
```



Additionally, I tested using vanilla numpy and `torch.tensor` by substituting with `o = torch.tensor(o, requires_grad=False)`. The warning doesn't occur in this case, but still, adjoint and backprop give different results.



👍 1



CatalinaAlbornoz on Apr 22, 2022

Contributor ...

Hi @vbelis, thank you for providing this information. We're looking into why this is happening. We'll keep you posted on any updates.



👍 1



antalszava on Apr 29, 2022

Contributor ...

Using parameter-shift matches the backprop values. This is indeed an issue with adjoint.



👍 1



antalszava mentioned this on May 6, 2022

🔗 [Fix diff_method="adjoint" bug with qml.Hermitian \(inc. adding tape.get_parameters . operations_only arg\) #2543](#)



antalszava closed this as [completed](#) in [#2543](#) on May 10, 2022

Fix `diff_method="adjoint"` bug with `qml.Hermitian` (inc. adding `tape.get_parameters` `operations_only` arg) #2543

<> Code

Merged antalszava merged 12 commits into `master` from `fix_adjoint_hermitian` on May 10, 2022

Conversation 13 Commits 12 Checks 0 Files changed 6 +148 -7



antalszava commented on May 6, 2022 · edited

Contributor

Context:

`diff_method="adjoint"` is a device differentiation method and the main logic is implemented in the `QubitDevice.adjoint_jacobian` method. The implementation depends on a private `tape._par_info` dictionary and the `tape.trainable_params` list of trainable parameter indices.

The implementation doesn't, however, consider the case when measurements contain parametrized observables such as `qml.Hermitian` that are different than the disallowed `qml.Hamiltonian` observable.

Description of the Change:

- Add a new `circuit_only` argument to the `tape.get_parameters` method such that parameters affecting the circuit can be obtained and parameters to measured observables can be excluded;
- Changes the logic of `adjoint_jacobian` such that it filters out entries from `tape._par_info` that are possible coming from measurements (checked via `hasattr(tape._par_info[k]["op"], "return_type")`);
- Warns if parameters to measured observables were marked as trainable with the adjoint diff method;
- Updates the doc of `adjoint_jacobian` such that the page mentions that no parametrized observable is supported.

Benefits:

- [\[BUG\] Adjoint and backpropagation differentiation gives different results for custom qml.Hermitian observables #2434](#) is solved:

Average loss over epoch 20: 0.0661188588
<Output tensor as expected>
Accuracy: 97.5%



- The case of computing gradients of circuits with parametrized measured observables is cleared in the docs.

Possible Drawbacks:

N/A

Related GitHub Issues:

Closes [#2434](#).



Reviewers

rmoyard ✓

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

[\[BUG\] Adjoint and backpropagation different...](#)

Notifications

Customize

[Subscribe](#)

You're not receiving notifications from this thread.

2 participants

